CIPHER: Contextual Instruction Precedence and Hierarchy Evaluation for Repositories

Research Proposal

Anonymous Authors

***Abstract*—Repository instruction files guide AI coding agents through project conventions, but apparent disagreement does not always indicate a defect. Two requirements may concern different directories, apply under different conditions, or be resolved by a documented local override. We propose CIPHER, Contextual Instruction Precedence and Hierarchy Evaluation for Repositories, a diagnostic framework for evaluating this distinction. CIPHER treats a case as two source-grounded instruction spans together with their document context, repository revision, target task, file paths, and an explicit instruction-loading profile. Its annotation scheme separates unresolved conflicts, valid overrides, disjoint applicability, compatible requirements, and insufficient evidence. The initial scope is root and nested AGENTS.md files under a frozen directory-precedence profile. We describe a study that combines naturally occurring instruction pairs with controlled variants that preserve rule wording while changing applicability or precedence, alongside transformations that should preserve the answer. Two independent annotators and an adjudicator will establish reference labels. A structured applicability-and-resolution pipeline will be compared with heuristic and direct language-model baselines, including a strong baseline receiving the same contextual evidence. Proposed measures include conflict precision and recall, false alarms on valid overrides, correctness across paired variants, and evidence fidelity. This paper contributes the task definition, annotation scheme, and evaluation protocol; it does not report a completed dataset or experimental results.**

***Index Terms*—AI coding agents; AGENTS.md; instruction conflicts; repository mining; natural language software artifacts; scope and precedence**

I. Introduction

AI coding agents receive project guidance through natural-language repository files. The AGENTS.md format permits nested guidance and describes precedence for the file closest to an edited target [1]. Empirical research has characterized these artifacts as maintained sources of project instructions [2]. Their interpretation therefore depends on more than isolated sentence meaning: a reader must also identify the task, the relevant directory, and the authority of each instruction.

A defect detector can fail in two directions. It can overlook incompatible obligations, leaving an agent without a coherent instruction set. It can also report a legitimate exception as a defect and recommend removing useful local guidance. The latter matters when repositories contain multiple packages, testing environments, or language conventions. A statement about one package need not constrain another.

For an illustrative example, let rule A require exactly four spaces per indentation level and rule B require exactly two. If both occur in the same effective scope with equal authority, the requirements conflict for a newly indented line. If B belongs to a nested AGENTS.md that governs the target and takes precedence, the result is a valid override. If B belongs to an unrelated directory, the pair is not jointly applicable. These are authored examples, not observations from a mined repository.

CIPHER asks whether conflict detectors make these distinctions correctly. We define a task-conditioned labeling problem and a controlled evaluation protocol. The proposed contribution is a human-adjudicated semantic diagnostic resource and its comparative evaluation. No benchmark has yet been collected and no model performance is claimed. This proposal makes the evaluation criteria explicit before implementation or data collection.

II. Related Work and Research Gap

The closest detector study is Configuration Smells [3]. Its analysis of 100 repositories includes conflicting instructions; manual inspection confirmed 16 of 28 conflict flags, giving the reported 57% precision. This motivates examining false alarms, but does not show that scope or precedence caused those particular errors. Its prompt already includes guidance intended to reduce false positives.

WIRE [4] already grounds rules in source spans, models activation and decision surfaces, and evaluates witnessed collisions. Its study omits matched compatible-pair and priority-insertion controls. ContextCov [5] preserves contextual structure while translating instructions into executable checks. OctoBench [6] includes explicit conflicts and measures which source an agent follows without prescribing a priority order for that subset. CIPHER instead asks for a semantic classification under a declared repository profile.

Practical systems also constrain the novelty claim. Guidefold [7] documents scope authority and structural replays and already motivates independent semantic validation. Ruleherd [8] audits cross-file conflicts with keyword and polarity heuristics. None of these principles is claimed as new. CIPHER proposes their application to a five-way, task-conditioned classification problem for root and nested AGENTS.md at one revision, compared with direct models receiving the same evidence.

III. Task Definition and Instruction Profile

A case contains a repository revision, two verbatim rule spans with headings, source paths, a task and target path, and a versioned loading profile. Packets also include relevant exception and priority statements outside the selected spans. Missing decisive context yields insufficient evidence or a documented

TABLE I
CASE LABELS UNDER THE FROZEN INSTRUCTION PROFILE

Label	Meaning
Unresolved conflict	Both hard requirements remain effective for the same decision and cannot jointly be satisfied; the declared profile supplies no valid winner.
Valid override	Both rules apply before priority is considered, but a documented precedence rule resolves their incompatible requirements for this task.
Disjoint applicability	Before considering priority, at least one rule is inactive because of its directory, task, environment, or explicit condition.
Compatible	The effective requirements can be satisfied together, including rules about different decisions and nonbinding preferences.
Insufficient evidence	The available material cannot establish meaning, applicability, or precedence well enough to select another label.

exclusion. The label concerns this pair and task, not overall repository quality. Source anchors establish provenance rather than correctness.

The initial profile includes only root and nested AGENTS.md files. It gathers the ancestor chain for a single target and preserves conditions attached to each rule. A more specific applicable file can override an incompatible ancestor instruction in accordance with the declared profile; unrelated ancestor guidance remains in force. Equal-authority contradictions are not resolved by arbitrary presentation order. This operational interpretation is documented and frozen for the experiment, using the format guidance as its basis [1]. It is not a claim that every coding tool implements an identical loader.

The five labels in Table I separate incompatibility from applicability and authority. Annotation checks evidence sufficiency, joint applicability before priority, mutual satisfiability, and then any documented override. An explicit condition that excludes the task makes a rule inactive; priority instead resolves rules that would otherwise both apply. Thus, a superseded ancestor is not mislabeled as disjoint. A soft preference is not strengthened into a prohibition. Examples and descriptions are not binding unless their context makes them normative.

IV. PROPOSED DIAGNOSTIC PIPELINE

The prototype will preserve Markdown structure and extract candidate claims with exact source spans. Each claim record will include its action, object, modality, explicit condition, document heading, and source path. Candidate generation will use shared entities and decision topics, supplemented by random pairs and exhaustive review of selected small trees so that retrieval misses can be inspected.

A deterministic component will evaluate directory ancestry and the frozen profile. A language-model component will assess task conditions and whether the surviving requirements regulate the same decision incompatibly. The result will include a label, both source anchors, the effective scope, the rule authorizing any override, and a concise justification. Missing evidence will be retained rather than completed by assumption.

The pipeline is a reference implementation for evaluating the task, not a claim to a new general reasoning architecture. It will not automatically repair instructions or execute repository commands. In the core study, README, CONTRIBUTING, CLAUDE.md, and SKILL.md are outside the binding instruction hierarchy. Extensions must specify their loading and authority independently instead of inventing a universal order among filenames.

V. PLANNED EVALUATION

RQ1 asks how reliably annotators distinguish the five labels in the sampled material. RQ2 asks whether the structured pipeline improves conflict classification relative to a strong direct model given the same evidence. RQ3 asks whether predictions change correctly under relevant context changes and remain correct under irrelevant presentation changes. These questions concern diagnostic validity and classification, not downstream coding productivity.

Data and sampling. Existing configuration corpora [2], [9] can seed repository discovery. We will record the release, eligibility rules, search date, licenses, exclusions, and exact repository commits. Eligibility requires accessible root and nested instructions and enough context to specify a single-target task. Forks and copied instruction templates will be grouped before splitting. Original files will remain unmodified in a separate evidence archive.

A feasible pilot target is eight calibration source pairs from two repositories and 32 evaluation source pairs from eight different repositories. Each evaluation family will retain its original context and a validated answer-preserving control. Up to 32 answer-changing contexts will be added where independently valid transformations exist, giving a target of 64 to 96 evaluation contexts. These are proposed counts, not collected data or a power calculation. Original cases will not be discarded merely because they lack a valid answer-changing transformation.

Original contexts preserve genuine source text and paths; their concrete tasks may be authored rather than observed agent sessions. Task origin and construction will be recorded. Interventions preserve rule wording while changing a documented property such as file placement or target path. Controls may reorder evidence or alter harmless formatting. Only human-verified transformations will be admitted. Original and transformed results will be separate; neither this selected corpus nor its transformed subset estimates population prevalence. Sparse natural conflicts will remain an explicit limitation.

Reference labels. Two authors will independently annotate each case without system predictions; the third will adjudicate disagreements. Calibration cases will be excluded from evaluation. The handbook, profile, prompts, selection rules, and metric definitions will be frozen before held-out runs. We will preserve initial judgments, agreement statistics, adjudication

reasons, and the supporting evidence. Gold labels will not be generated solely by an evaluated model or inferred from a merged code change.

Comparators. The primary arms are a heuristic, a direct contextual model, and the structured pipeline. The latter two use one fixed model and the same complete evidence packet, with declared token and call budgets. Ruleherd [8] requires an audited mapping of repository flags to case pairs; a reimplementation must be named an attributed adaptation. Its binary outputs receive conflict metrics only. The published smell prompt [3] is an optional comparison. A text-only condition is an information ablation, not evidence of method superiority.

Measures and analysis. We will report conflict precision, recall, and F1, five-class performance with class support, false-conflict rates on overrides and disjoint cases, and exact correctness on both members of each answer-changing pair. Answer-preserving controls require correctness and stability: identical wrong predictions are not success. Evidence anchors and semantic justifications are audited separately. Voluntary abstention differs from a gold insufficient-evidence label. It is incorrect in five-class accuracy and misses a reference conflict for binary recall; class-specific abstention rates are also reported. Versions, prompts, decoding settings, repetitions, and inference costs will be logged.

Uncertainty will be summarized with repository-level clusters rather than treating transformed siblings as independent. Any family-level interval is conditional on the sampled repositories. With only eight evaluation repositories, raw counts, paired differences, and case analysis carry more weight than interval precision. Retrieval coverage is reported only where a complete pair inventory was reviewed; candidate-level recall is not repository-wide recall.

VI. Limitations and Expected Research Value

The current contribution is a proposal. Its central hypothesis, that explicit scope and precedence improve semantic conflict judgments, remains untested. Human agreement may itself be low, and a detector could reduce false alarms by missing true conflicts. Reporting recall, class support, and abstention alongside false alarms prevents that trade-off from being hidden.

A single profile and a small corpus limit generalization. Public repositories may also appear in model pretraining, and transformed cases may contain authoring artifacts. Repository-disjoint splits, template grouping, blinded annotation, and separate natural-case results reduce but cannot eliminate those risks. A useful negative outcome would identify an unreliable label boundary or show that a strong direct prompt is sufficient.

Improved classification would not establish better coding-task success; context-file effectiveness requires its own evaluation [10]. Historical staleness is a separate problem studied in Context Rot [11] and requires revision evidence. Future work may examine additional loaders, multi-target tasks, temporal changes, and developer review, after the diagnostic task is validated.

VII. Conclusion

CIPHER proposes a focused evaluation of when repository instructions genuinely conflict. By keeping rule wording fixed while controlling task context and authority, the study can test whether detectors understand legitimate exceptions. The immediate next step is an independently annotated pilot with strong contextual baselines and complete evidence records.

Acknowledgment of AI Assistance

OpenAI ChatGPT assisted with concept exploration, literature synthesis, drafting the abstract and all main sections, and document preparation. No experimental measurements are reported. The named authors are responsible for reviewing the manuscript and for conducting and verifying any subsequent research.

References

[1] AGENTS.md, "AGENTS.md format and nested instruction guidance," Online specification, accessed 18 October 2026. [Online]. Available: https://agents.md/

[2] W. Chatlatanagulchai et al., "Agent READMEs: An Empirical Study of Context Files for Agentic Coding," arXiv:2511.12884v2, 2025, revised 2026. [Online]. Available: https://arxiv.org/abs/2511.12884v2

[3] H. V. F. dos Santos, V. Costa, J. E. Montandon, L. L. Silva, and M. T. Valente, "Configuration Smells in AGENTS.md Files: Common Mistakes in Configuring Coding Agents," SCAM 2026; arXiv:2606.15828v5. [Online]. Available: https://arxiv.org/abs/2606.15828v5

[4] L. Yan, X. Chen, and X. Zhang, "WIRE: Profiling Witnessed Within-Policy Instruction Collisions in LLM Agents," arXiv:2605.27784v2, 2026. [Online]. Available: https://arxiv.org/abs/2605.27784v2

[5] R. K Sharma, "ContextCov: Bridging the Gap Between Developer Intent and Autonomous Agent Execution," arXiv:2603.00822v2, 2026. [Online]. Available: https://arxiv.org/html/2603.00822v2

[6] D. Ding et al., "OctoBench: Benchmarking Scaffold-Aware Instruction Following in Repository-Grounded Agentic Coding," ACL 2026, pp. 5958-5978. [Online]. Available: https://aclanthology.org/2026.acl-long.269/

[7] wiatrM, "Guidefold research notes," Software project documentation, accessed 18 October 2026. [Online]. Available: https://github.com/wiatrM/guidefold/blob/main/docs/RESEARCH.md

[8] princejain756, "Ruleherd," Software project documentation, accessed 18 October 2026. [Online]. Available: https://github.com/princejain756/ruleherd

[9] M. Galster, S. Mohsenimofidi, L. Bohme, J. L. Lulla, M. A. Abubakar, C. Treude, and S. Baltes, "A Dataset of Agentic AI Coding Tool Configurations," AIware 2026; arXiv:2605.08435. [Online]. Available: https://arxiv.org/abs/2605.08435

[10] T. Gloaguen, N. Mundler-Sasahara, M. N. Muller, V. Raychev, and M. Vechev, "Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents?," arXiv:2602.11988v3, 2026. [Online]. Available: https://arxiv.org/abs/2602.11988v3

[11] C. Treude and S. Baltes, "Context Rot in AI-Assisted Software Development: Repurposing Documentation Consistency for AI Configuration Artifacts," arXiv:2606.09090, 2026. [Online]. Available: https://arxiv.org/abs/2606.09090